

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create a simple grayscale gradient image for demonstration
width, height = 256, 128
gradient_image = np.zeros((height, width), dtype=np.uint8)
for i in range(width):
    gradient_image[:, i] = int(i)

# Display the original gradient image
plt.figure(figsize=(10, 5))
plt.imshow(gradient_image, cmap='gray')
plt.title('Original Grayscale Gradient Image (256 levels)')
plt.axis('off')
plt.show()

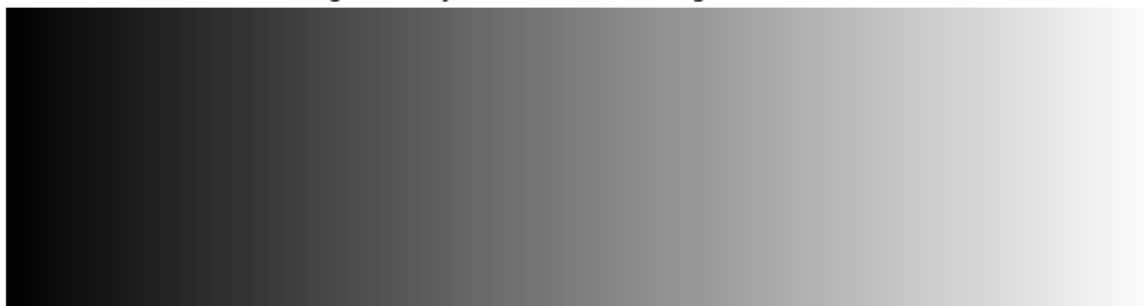
# Define different numbers of quantization levels to demonstrate the effect
quantization_levels = [64, 32, 16, 8, 4, 2]

quantized_images = {}
for levels in quantization_levels:
    # Calculate the step size for quantization
    # For N levels, each step covers 256/N intensity values
    step_size = 256 // levels

    # Apply quantization:
    # 1. Divide by step_size: this maps the 0-255 range to 0 to (levels-1)
    # 2. Multiply by step_size: this maps these discrete values back to the original range
    # The // operator performs integer division
    quantized_img = (gradient_image // step_size) * step_size
    quantized_images[levels] = quantized_img

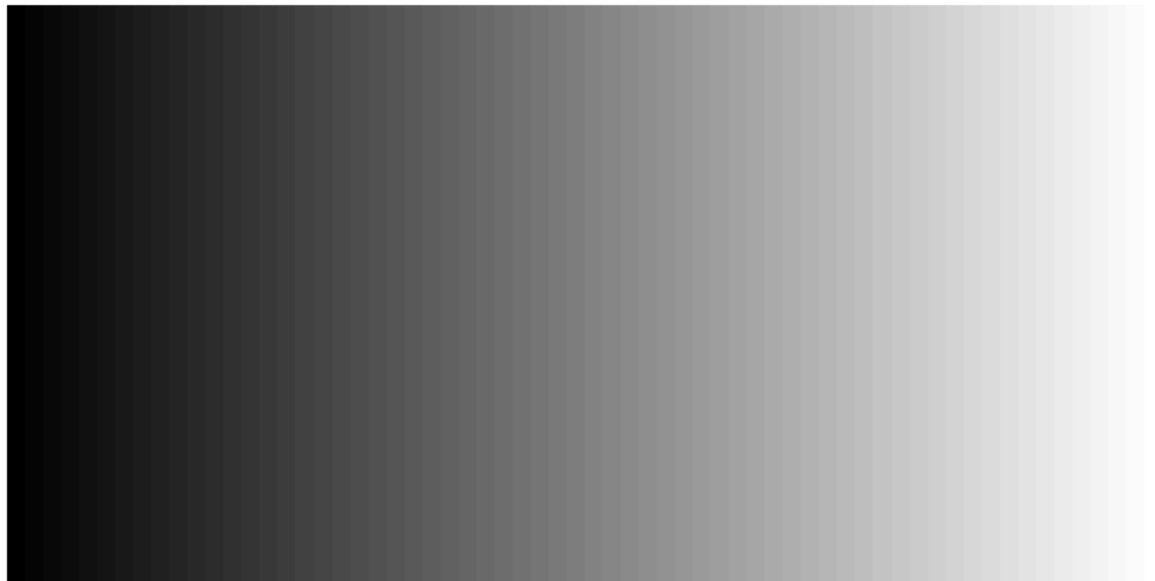
# Display each quantized image
plt.figure(figsize=(10, 5))
plt.imshow(quantized_img, cmap='gray')
plt.title(f'Quantized Image ({levels} levels)')
plt.axis('off')
plt.show()
```

Original Grayscale Gradient Image (256 levels)

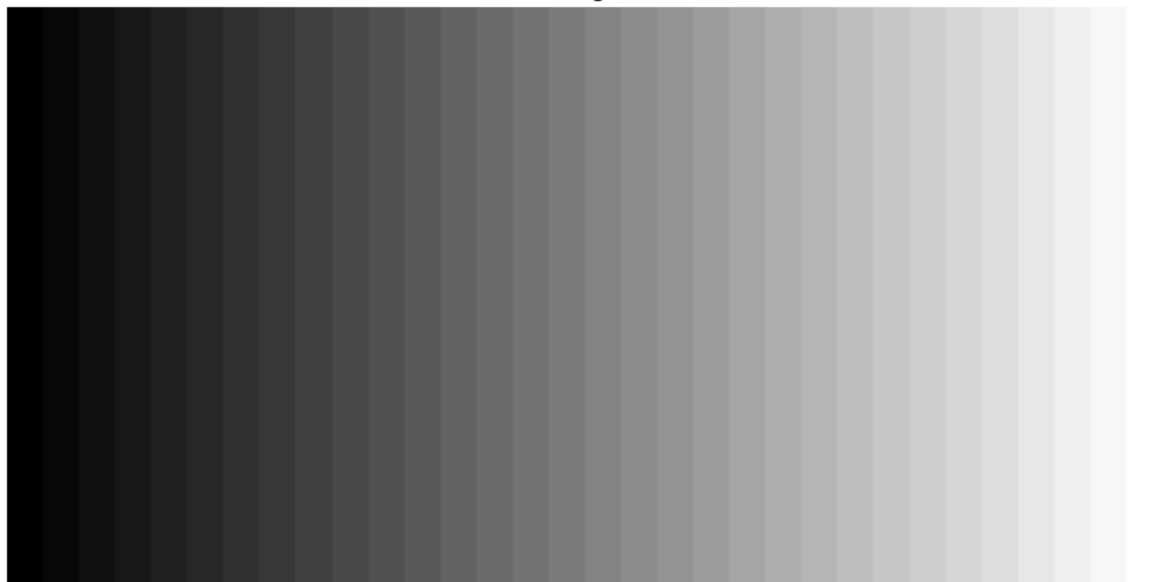




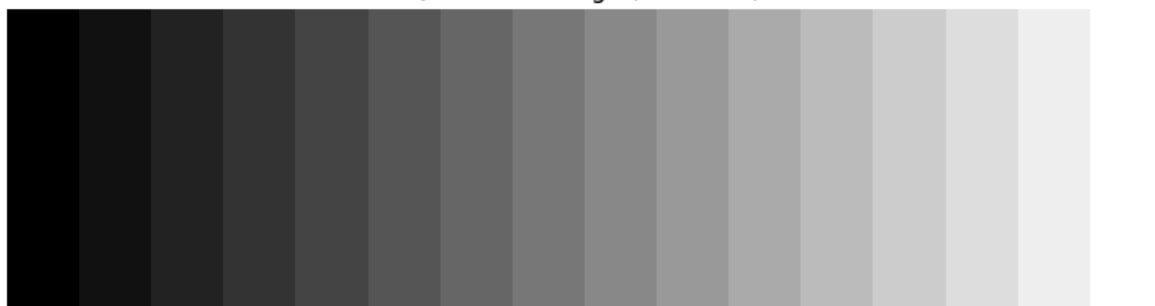
Quantized Image (64 levels)



Quantized Image (32 levels)



Quantized Image (16 levels)





Quantized Image (8 levels)



Quantized Image (4 levels)



Quantized Image (2 levels)

